

The Git Workflow

Using a Git repo involves the following steps. First, you create the Git repo by running the `git init` or `git clone` command. Second, you add files with the `git add <filename>` command for the repo to track. Third, once you've added files, you can commit them with the `git commit -am "<descriptive commit message>"` command. At this point, you're ready to make more changes to your code.

You can view the help file for each of these commands by running `git help <command>`, such as `git help init` or `git help add`. These help pages are handy for reference, although they're too dry and technical to use as tutorials. You'll learn more details about each of these commands later, but first, you need to understand a few Git concepts to make the rest of this chapter easier to digest.

How Git Keeps Track of File Status

All files in a working directory are either tracked or untracked by Git. *Tracked files* are the files that have been added and committed to the repo, whereas every other file is *untracked*. To the Git repo, untracked files in the working copy might as well not exist. On the other hand, the tracked files exist in one of three other states:

- The *committed state* is when a file in the working copy is identical to the repo's most recent commit. (This is also sometimes called the *unmodified state* or *clean state*.)
- The *modified state* is when a file in the working copy is different than the repo's most recent commit.
- The *staged state* is when a file has been modified and marked to be included in the next commit. We say that the file is *staged* or *in the staging area*. (The staging area is also known as the *index* or *cache*.)

[Figure 12-4](#) contains a diagram of how a file moves between these four states. You can add an untracked file to the Git repo, in which case it becomes tracked and staged. You can then commit staged files to put them into the committed state. You don't need any Git command to put a file into